

OTIF Performance Intelligence Engine — Complete Build Workflow

Full technical documentation, from raw system data to interactive prototype

This document is written to be self-contained — someone with no prior context on our discussions should be able to read this and understand exactly what we're building, why each piece exists, and how to build it.

1. What We're Building — The Problem in One Paragraph

On-Time-In-Full (OTIF) delivery performance is a critical supply chain reliability metric. Missed OTIF can originate from failures across order capture, inventory availability, warehouse operations, vendor performance, distribution center constraints, transportation, or customer delivery requirements — but companies typically only discover an OTIF failure *after* it happens, because there's no structured way to see it coming. This project builds an AI-enabled system that: (1) explains *why* past OTIF misses happened, by mechanically deriving root causes from system data rather than relying on manual tagging; (2) predicts which *currently open* orders are at risk of missing OTIF before they actually fail; (3) explains *why* each at-risk order is flagged, in plain language; and (4) recommends a mitigation action — while accounting for the fact that fixing one order's problem can create a new problem for another order competing for the same resources.

2. Data Foundation — The Multi-Table Structure

Why multiple tables, not one flat sheet: Orders relate to vendors, DCs, carriers, and customers in "one-to-many" ways — one vendor supplies hundreds of orders, one DC handles thousands. Flattening everything into a single sheet means repeating vendor/DC/customer details on every row, which creates massive redundancy and makes updates unreliable (update a vendor's stats in one place, and it's now wrong everywhere else it was copied). The correct structure is a **star schema**: one central fact table (what you're measuring) surrounded by dimension tables (reference data).

Order Lines (the FACT table — one row per SKU per order)

Grain: one row = one SKU on one order (not one row per order) — because a multi-SKU order can partially succeed and partially fail; each SKU needs its own outcome.

Core fields: order_line_id, order_id, sku_id, customer_id, vendor_id, dc_id, carrier_id/lane_id, order_date, promised_delivery_date, actual_delivery_date, ordered_qty, delivered_qty, order_value, inventory_on_hand_at_order, safety_stock_level, order_modified_flag, delivery_window_hours, root_cause_category, root_cause_detail, secondary_causes[], confidence, vendor_fault, otif_flag.

Additional fields required for root-cause derivation (these must be added to the generator — not all were in the original schema)

From ERP/CRM (Order Capture): order_creation_timestamp, order_cutoff_time, credit_hold_flag, credit_release_timestamp, customer_po_qty, customer_master_complete_flag, atp_promised_date, atp_system_calculated_feasible_date

From WMS (Warehouse Ops): pick_task_start, pick_task_end, pick_sla_minutes, pick_line_qty, pack_start, pack_end, pack_sla_minutes, wms_ready_timestamp, wms_released_to_shipping_timestamp, dock_appointment_overlap_flag

From ERP/WMS (Inventory + DC): wms_cycle_count_variance_flag, allocation_conflict_flag, dc_assigned, dc_nearest_with_stock, dc_daily_order_volume, dc_rated_capacity

From SRM/ERP (Vendor): po_promised_date, po_qty, grn_actual_date, grn_qty, qc_rejection_flag, vendor_contracted_leadtime, vendor_actual_leadtime, po_creation_date, required_by_date

From TMS (Transport): tender_timestamp, tender_cutoff, carrier_acceptance_flag, scheduled_pickup, actual_pickup, planned_transit_time, actual_transit_time, delivery_attempt_exception_code

From CRM/TMS (Customer): customer_reschedule_timestamp, original_delivery_date, appointment_slot_conflict_flag, address_validation_failure_flag

Grain note: some of these fields are genuinely order-level (credit hold, DC assignment, ATP routing) even though the fact table is line-level — order-level outcomes must be broadcast to

every line of that order; line-level outcomes (pick errors, short-shipment quantities) apply only to their own line. Tag each rule by grain before implementation to avoid inconsistent joins on multi-SKU orders.

Dimension tables

- **Vendor Master** (one row per vendor): vendor_id, vendor_name, vendor_rolling_otif_30d, vendor_rolling_otif_90d, vendor_lead_time_avg, vendor_lead_time_stddev, vendor_capacity_utilization, vendor_category. **Critical rule:** vendor_rolling_otif_30d/90d must be calculated *excluding* orders where vendor_fault == False (i.e., exclude "PO issued late by buyer" cases) — otherwise you penalize vendors for your own procurement team's mistakes, and your vendor risk scores become systematically unfair.
- **DC Master** (one row per DC): dc_id, dc_location, dc_capacity. dc_current_backlog_pct should ideally be its own time-series table (dc_id, date, backlog_pct) since it changes daily.
- **Carrier/Lane Master** (one row per origin-destination-carrier combo): lane_id, origin, destination, carrier_name, lane_carrier_punctuality_90d — kept lane-specific, not carrier-wide, because the same carrier can be reliable on one route and unreliable on another.
- **Customer Master** (one row per customer): customer_id, customer_tier, penalty_sensitivity.
- **Root Cause Reference** (one row per category): root_cause_code, recommended_action.

Two distinct versions of the Order Lines dataset

1. **Historical/closed orders** — fully populated including actual_delivery_date, delivered_qty, root_cause_category, otif_flag. This is your **training data**.
2. **Open/in-flight orders** — every feature populated except outcome fields, which are unknown. This is your **live scoring data**.

Conflating these breaks the model — you cannot train on data you're simultaneously trying to predict.

3. Phase 0 — Data Validation

What to do: Before any modeling, audit the synthetic dataset the way you'd audit a real one: referential integrity (every `vendor_id/dc_id/line_id` in Order Lines actually exists in its dimension table), logical consistency (`actual_delivery_date` never before `order_date`, `delivered_qty <= ordered_qty` unless over-delivery is intentionally modeled), and realistic class balance (OTIF success typically 80-95% in reality — don't generate 50/50).

Why: Undetected generator errors silently corrupt every downstream step; this is the step most teams skip under time pressure and the one that causes the most wasted debugging later.

Tool: pandas — `.isnull().sum()`, `.describe()`, targeted assert statements.

4. Phase 1 — Root-Cause Derivation Engine (Multi-Cause Version)

Why this exists at all: Real organizations do not have a human manually tagging every failed delivery with its root cause. What exists instead is fragmented system data (ERP order timestamps, WMS pick/pack events, SRM PO-to-GRN records, TMS shipment tracking) that has to be mechanically cross-referenced to *derive* the cause. Building this as deterministic rule logic — rather than assuming labels already exist — is both more realistic and more defensible when asked "where did your labels come from."

The 7-category taxonomy (the modeling target)

Chosen deliberately as a rollup of a much longer granular list (~30 specific causes), because a classifier/BBN with 30 sparse classes is unworkable on a synthetic dataset of a few hundred/thousand rows, while 7 categories is manageable and still traceable directly to the case brief's own language:

1. **Order Capture / Processing Delay**
2. **Vendor / Supplier Failure** (with a `vendor_fault` boolean sub-flag — distinguishing true vendor lateness from a buyer issuing the PO too late)
3. **Inventory Shortage / Availability**
4. **DC Capacity Constraint**
5. **Warehouse Operations Delay**
6. **Transportation / Carrier Delay**
7. **Customer Delivery / Appointment Issue**

Each historical order also carries `root_cause_detail` — the specific granular rule that fired (e.g., "Vendor short-shipment" under category 2) — used later to make explanation narratives specific rather than generic.

Step 1: Run every category's rules against every order — don't stop at the first match

The original design was a waterfall that stopped at the first rule fired, which meant an order with *both* a vendor delay and a DC congestion issue only ever got labeled with one cause, silently losing the second signal. The corrected version evaluates **all 7 categories' full rule sets for every order** and records everything that fires:

```
fired_causes = []
```

```
# Category 1 — Order Capture
```

```
if order_creation_timestamp > order_cutoff_time:
```

```
    fired_causes.append(("ORDER_CAPTURE", "Missed cutoff", confidence_flag))
```

```
if credit_hold_flag == True and credit_release_late:
```

```
    fired_causes.append(("ORDER_CAPTURE", "Credit hold released late", confidence_flag))
```

```
if order_modified_flag == True and (promised_delivery_date - modification_timestamp) <
minimum_lead_days:
```

```
    fired_causes.append(("ORDER_CAPTURE", "Late order amendment", confidence_flag))
```

```
if customer_po_qty != order_line_qty:
```

```
    fired_causes.append(("ORDER_CAPTURE", "Order entry mismatch", confidence_flag))
```

```
if customer_master_complete_flag == False:
```

```
    fired_causes.append(("ORDER_CAPTURE", "Incomplete customer master data",
confidence_flag))
```

```
if atp_promised_date < atp_system_calculated_feasible_date:
```

```
    fired_causes.append(("ORDER_CAPTURE", "Unrealistic promised date (ATP override)",
confidence_flag))
```

```
# Category 2 — Vendor / Supplier Failure
```

```
if grn_actual_date > po_promised_date and po_creation_date <= (required_by_date -
vendor_contracted_leadtime):
```

```
    fired_causes.append(("VENDOR_FAILURE", "Vendor late delivery to DC", confidence_flag))
```

```
# vendor_fault = True
```

```
if grn_actual_date > po_promised_date and po_creation_date > (required_by_date -
vendor_contracted_leadtime):
```

```
    fired_causes.append(("VENDOR_FAILURE", "PO issued late by buyer", confidence_flag))
```

```
# vendor_fault = False
```

```
if grn_qty < po_qty:
```

```
    fired_causes.append(("VENDOR_FAILURE", "Vendor short-shipment", confidence_flag))
```

```
# vendor_fault = True
```

```
if qc_rejection_flag == True:
```

```
    fired_causes.append(("VENDOR_FAILURE", "Vendor quality rejection at inbound QC",
confidence_flag)) # vendor_fault = True
```

```
if vendor_actual_leadtime > vendor_contracted_leadtime (recurring across recent POs):
```

```
fired_causes.append(("VENDOR_FAILURE", "Vendor lead-time variability vs. contract",  
confidence_flag)) # vendor_fault = True
```

```
# Category 3 — Inventory Shortage / Availability
```

```
if inventory_on_hand_at_order == 0 or inventory_on_hand_at_order < ordered_qty:  
    fired_causes.append(("INVENTORY_SHORTAGE", "Stockout at fulfilling location",  
confidence_flag))  
if wms_cycle_count_variance_flag == True:  
    fired_causes.append(("INVENTORY_SHORTAGE", "Inventory record inaccuracy",  
confidence_flag))  
if inventory_on_hand_at_order < safety_stock_level:  
    fired_causes.append(("INVENTORY_SHORTAGE", "Safety stock breached",  
confidence_flag))  
if allocation_conflict_flag == True:  
    fired_causes.append(("INVENTORY_SHORTAGE", "Stock allocated to conflicting  
order/channel", confidence_flag))
```

```
# Category 4 — DC Capacity Constraint
```

```
if dc_assigned != dc_nearest_with_stock:  
    fired_causes.append(("DC_CAPACITY", "Wrong fulfillment DC selected (ATP routing issue)",  
confidence_flag))  
if dc_daily_order_volume > dc_rated_capacity:  
    fired_causes.append(("DC_CAPACITY", "DC throughput ceiling breached", confidence_flag))  
if dock_appointment_overlap_flag == True:  
    fired_causes.append(("DC_CAPACITY", "Staging/dock congestion", confidence_flag))
```

```
# Category 5 — Warehouse Operations Delay
```

```
if (pick_task_end - pick_task_start) > pick_sla_minutes:  
    fired_causes.append(("WAREHOUSE_OPS", "Picking delay", confidence_flag))  
if pick_line_qty != order_line_qty:  
    fired_causes.append(("WAREHOUSE_OPS", "Picking error", confidence_flag))  
if (pack_end - pack_start) > pack_sla_minutes:  
    fired_causes.append(("WAREHOUSE_OPS", "Packing delay", confidence_flag))  
if (wms_released_to_shipping_timestamp - wms_ready_timestamp) > handoff_sla_minutes:  
    fired_causes.append(("WAREHOUSE_OPS", "WMS→TMS handoff failure",  
confidence_flag))
```

```
# Category 6 — Transportation / Carrier Delay
```

```
if tender_timestamp > tender_cutoff:  
    fired_causes.append(("TRANSPORT", "Shipment not tendered to carrier by cutoff",  
confidence_flag))  
if carrier_acceptance_flag == False:  
    fired_causes.append(("TRANSPORT", "Carrier capacity shortage / tender rejected",  
confidence_flag))
```

```

if actual_pickup > scheduled_pickup + pickup_tolerance:
    fired_causes.append(("TRANSPORT", "Missed carrier pickup", confidence_flag))
if actual_transit_time > planned_transit_time * variance_threshold:
    fired_causes.append(("TRANSPORT", "In-transit delay", confidence_flag))
if delivery_attempt_exception_code == "ACCESS_ISSUE":
    fired_causes.append(("TRANSPORT", "Last-mile delivery failure", confidence_flag))

# Category 7 — Customer Delivery / Appointment Issue
if delivery_attempt_exception_code == "CUSTOMER_UNAVAILABLE":
    fired_causes.append(("CUSTOMER_DELIVERY", "Customer unavailable at delivery
window", confidence_flag))
if customer_reschedule_timestamp is not null and customer_reschedule_timestamp >
order_dispatch_timestamp:
    fired_causes.append(("CUSTOMER_DELIVERY", "Customer-requested reschedule after
dispatch", confidence_flag))
if appointment_slot_conflict_flag == True:
    fired_causes.append(("CUSTOMER_DELIVERY", "Delivery appointment conflict",
confidence_flag))
if address_validation_failure_flag == True:
    fired_causes.append(("CUSTOMER_DELIVERY", "Incorrect delivery address/access
instructions", confidence_flag))

```

Important nuance: if multiple rules within the *same* category fire together (e.g., both "picking delay" and "picking error" under Warehouse Ops), both still get appended to `fired_causes`. Pick the one that fired chronologically first as that category's representative detail; push the other into `secondary_causes` under the same category rather than silently dropping it.

Step 2: Assign primary vs. secondary

If `fired_causes` is empty → `root_cause_category` = "UNKNOWN", `confidence` = "LOW" — keep this bucket genuine; real data always has ambiguous cases, and forcing 100% classification looks naive to a judge familiar with this space.

If one or more entries exist, apply the priority order (upstream causes take precedence, on the logic that fixing the earliest failure in the chain would likely have prevented downstream ones too):

```

PRIORITY_ORDER = ["ORDER_CAPTURE", "VENDOR_FAILURE",
"INVENTORY_SHORTAGE",
"DC_CAPACITY", "WAREHOUSE_OPS", "TRANSPORT",
"CUSTOMER_DELIVERY"]

```

`primary_category` = earliest entry in `PRIORITY_ORDER` present in `fired_causes`

primary_detail = that category's specific fired rule text
secondary_causes = every other fired entry, excluded from primary

Final output structure per historical order:

```
{  
  order_id, root_cause_category, root_cause_detail,  
  secondary_causes: [{category, detail, confidence}, ...],  
  confidence,      # of the primary cause specifically  
  vendor_fault     # boolean, relevant when primary or any secondary = VENDOR_FAILURE  
}
```

Confidence flag logic (borrowed from source research on realistic enterprise data gaps)

IF all fields required for the fired rule are non-null and pass sanity checks → confidence = "HIGH"

ELIF the rule fired using a proxy/partial field (e.g., DC throughput inferred without labor data) → confidence = "MEDIUM"

ELSE (sparse/incomplete data, or fell to UNKNOWN) → confidence = "LOW"

This becomes a genuinely strong line in later explanation cards: *"Root cause: Vendor short-shipment (High confidence — PO/GRN data complete)"* vs. *"Root cause: likely DC delay (Medium confidence — WMS labor data unavailable)"* — a materially stronger pilot story than silently guessing when data is thin.

Critical scope boundaries (state these explicitly, don't paper over them)

- **This phase only runs on CLOSED orders** — it requires `actual_delivery_date`, `grn_actual_date`, etc., which don't exist for orders still in flight. **Open orders are never scored through this rule engine directly** — they're scored by the predictive model (Phase 4/7), and only get their *actual* root cause derived once they eventually close, which is what feeds the retraining loop (Phase 12).
- **True stockout upstream cause** (was it a forecasting miss, a vendor delay, or a planning parameter error?) genuinely needs a demand-planning/APS system this pipeline doesn't have access to. State plainly that this pipeline identifies *that* a shortage occurred and *where*, but not always *why it originated upstream* — a legitimate, defensible scope boundary rather than something to paper over.
- **What was deliberately excluded from the taxonomy and why:** Demand Planning/Forecasting (lives in a separate APS tool, not in ERP/WMS/TMS/SRM/CRM — flagged as a named future-phase gap, not built for); Master Data/Systems Integration Lag (folded into "WMS→TMS handoff failure," which is directly observable, rather than

treated as its own category); External/Force Majeure — weather, strikes, customs (none of the five source systems capture this; kept as a fallback to the UNKNOWN bucket rather than pretending it's detectable from ERP data).

5. Phase 2 — Feature Engineering

What to do: Merge the normalized multi-table structure into one wide, flat table via `.merge()` — one row per order line, with every feature computed and attached:

`inventory_coverage_days`, `vendor_rolling_otif_30d`,
`vendor_lead_time_variance`, `dc_current_backlog_pct`,
`lane_carrier_punctuality_90d`, `order_size_vs_avg`, `delivery_window_hours`,
`days_to_promised_date`, `sku_stockout_history_90d`,
`order_modified_after_entry`.

The data leakage problem — the most important thing in this phase:

`vendor_rolling_otif_30d` for an order placed on March 15th must only use vendor performance data from *before* March 15th. Calculating it from the vendor's full-year average (including data from after the order) leaks future information into training data — the model looks artificially accurate in testing, then fails on real forward predictions, because in production you never have "future" data for an open order.

Additional feature requirement — secondary causes as model inputs: add binary flag columns derived from each historical order's `secondary_causes` list (e.g., `has_secondary_dc_capacity`, `has_secondary_warehouse_ops`) as **input features**, not targets. This is the entire point of building multi-cause detection in Phase 1 — an order with both vendor delay AND DC congestion is a genuinely different, harder case than one with vendor delay alone, and the model needs that structure to learn from it. Skipping this step means Phase 1's multi-cause upgrade produces no actual benefit downstream.

Vendor metric fairness rule: when calculating `vendor_rolling_otif_30d/90d`, exclude orders where `vendor_fault == False`.

Open-order feature parity — an unresolved design decision requiring a team decision before scoring is built: secondary-cause-derived flag features only exist for *closed* orders, because they come from Phase 1's retrospective rules — an open order hasn't happened yet, so you don't know if a secondary cause will occur. You need forward-looking proxy versions instead — e.g., instead of the retrospective flag `has_secondary_dc_capacity`, use `dc_current_backlog_pct > threshold` as a leading indicator of the same underlying risk at scoring time. **You must build two versions of the feature table:** a training version (using retrospective flags, computed from Phase 1 output) and a scoring version (using only signals knowable before the order closes) — with columns that conceptually align even though they're

computed differently. This is the same data-leakage family of problem as the rolling-window issue above, just appearing in a new place because of the multi-cause upgrade. **This needs to be explicitly agreed with your teammates before the scoring pipeline is built**, not discovered after.

Tool: pandas — groupby + rolling window operations, strictly time-bounded.

6. Phase 3 — Time-Based Train/Test Split

What to do: Split chronologically — train on the first ~70-80% of the synthetic date range, test on the most recent ~20-30%. Not random.

Why: A random split lets the model see patterns from orders chronologically *after* the ones it's tested on, which doesn't reflect real deployment (always predicting forward, never backward). Worth naming explicitly if a judge asks about validation methodology.

7. Phase 4 — Train the XGBoost Classifier

What to do: Train a multi-class classifier predicting `root_cause_category` (7 classes) from the Phase 2 feature table. Evaluate using **recall, precision, and AUC-ROC** — not raw accuracy, which is misleading here since OTIF success is the majority class (a model that always predicts "success" could look 85-90% "accurate" while being useless).

Why recall matters most: missing a real at-risk order (false negative) costs an actual stockout/penalty; flagging a safe order as risky (false positive) only costs a planner a few minutes of review. This asymmetry means tuning the decision threshold to favor catching more real risks, even at the cost of more false alarms — and being able to justify that threshold choice explicitly rather than defaulting to 0.5.

Tool: xgboost, with scikit-learn for splitting, metrics, threshold tuning.

8. Phase 5 — Attach SHAP

What SHAP is: a post-hoc explainer, not a model itself — it sits on top of an already-trained model. Using game-theoretic Shapley values, it decomposes one specific prediction into the individual percentage contribution of each input feature (e.g., 78% risk = 35% base rate + 32%

from low inventory coverage + 21% from declining vendor OTIF + 14% from DC backlog – 9% from small order size).

Why it matters here specifically: this is where the percentage attribution the team originally wanted to hand-build actually belongs — generated automatically from real, trained-model feature contributions, never invented or hand-assigned in Phase 1. It's also the single most convincing artifact for the demo video, since it turns a bare probability into a defensible "here's exactly why."

What to build: `shap.TreeExplainer` on the trained model, generating SHAP values for every test-set and later every live open-order prediction; a small function formatting the top 3-4 contributing features per order into readable text.

Honest limitation: SHAP explains correlation, not causation — it tells you the model relied heavily on a feature, not that the feature *caused* the outcome in a structural sense. This is exactly the gap the Bayesian Network is meant to fill.

9. Phase 6 — Build the Bayesian Network

What to do: Define nodes matching the 7-category taxonomy (plus optionally a buyer-fault sub-node under Vendor Failure) feeding into an OTIF Outcome node, with directed edges reflecting the same causal/priority logic used in Phase 1. Populate conditional probability tables (CPTs) via parameter estimation from historical data (pgmpy's `BayesianEstimator`) rather than hand-guessing — this is the honest middle ground between full manual specification and full data-driven discovery, and structurally it's the same expert-elicitation-plus-data approach used in prior BBN/Monte Carlo risk work.

Why build this alongside XGBoost rather than instead of it: XGBoost + SHAP tells you *which features mattered* (correlational). A Bayesian Network tells you *how risk propagated through the system* (causal pathway) — e.g., "vendor delay increased DC backlog, which compounded the risk" — a richer, more intuitive story for a planner than a flat feature-importance list, and a genuine technical differentiator versus a SHAP-only pitch.

Multi-cause evidence — critical, easy to miss: when scoring an order, set evidence on **every node whose category appears in fired_causes** (both primary and secondary), not just the primary. A network that only ever receives one active node per order can't show meaningful propagation between factors — which defeats the entire purpose of having built multi-cause detection in Phase 1.

Tool: pgmpy — hand-defined structure (faster, more defensible at this data scale) plus `BayesianEstimator` for CPT learning; `HillClimbSearch/BicScore` available if you want

to demonstrate structure learning from data as a stated capability, even if not the primary approach used for the demo.

10. Phase 7 — Inference on Open Orders

What to do: For each open order, set evidence from its current known feature values, run inference to get the posterior probability of OTIF failure, broken down by which causal pathway contributed most.

On exact inference vs. Monte Carlo: at this network size (5-7 nodes), exact inference (VariableElimination) is sufficient — Monte Carlo sampling isn't strictly necessary here. Frame Monte Carlo explicitly as *how you'd scale inference for a much larger, more interconnected production network* (dozens of SKU/vendor/DC-specific nodes), not as something this demo-scale network required. Claiming Monte Carlo was needed for a 6-7 node network is the kind of overclaim a technical judge could catch — precision here builds credibility rather than costing you sophistication points.

Tool: pgmpy's VariableElimination for the demo; name its approximate/sampling inference methods as the stated production-scale path.

11. Phase 8 — Fusion Layer

What to do: Combine everything into one structured record per open order:

```
{
  order_id, xgb_risk_score, shap_top_factors[],
  bbn_posterior_probability, bbn_causal_pathway,
  combined_risk_score, confidence
}
```

Why: This is the actual product — a single coherent risk record the interactive tool queries. Keep the combination logic simple and explainable (e.g., a straightforward weighted average or "take the higher of the two") — stacking a second black-box model on top of two already-explainable ones would undermine the whole explainability story built up to this point.

Phase 8.3 — Mitigation Conflict Check

The problem this solves: a mitigation recommendation for one order (e.g., "reallocate from DC-B" or "expedite via alternate carrier") draws on a shared, finite resource — that DC's spare

stock, or that carrier's available capacity — that *other* currently at-risk orders may also be drawing on. Recommending the same fix independently for many orders without checking for overlap can solve one order's risk while silently creating a new one elsewhere.

What to do — deliberately scoped down from full multi-order optimization (a genuinely hard combinatorial problem real WMS/TMS vendors sell entire products to solve) to a lightweight conflict check:

```
proposed_action = get_recommendation(order)
resource_pool = identify_shared_resource(proposed_action) # e.g., DC-B's available stock,
Carrier X's capacity
concurrent_demand = SUM(reallocation requests already proposed for resource_pool, across
all currently at-risk orders)
```

IF `concurrent_demand > resource_pool.available_capacity`:

→ flag "CONTESTED" — do not auto-recommend; surface all competing orders together to the planner with a prioritization suggestion (typically by `priority_score`)

ELSE:

→ recommendation stands as-is

Why routing to a human on conflict is actually the stronger pitch, not a weaker one:

claiming full automation with zero human oversight is often less credible to judges than showing the system correctly recognizes the limits of automated resolution and escalates appropriately. For the demo, construct at least one deliberate example where two at-risk orders compete for the same DC's spare stock, and show the system correctly flags it as contested rather than double-recommending the same fix.

Phase 8.5 — High-Risk Entity Rollups

The gap this closes: everything up to this point scores individual *orders*. The case brief explicitly asks for aggregation to vendor, DC, lane, customer, SKU, and order-type level — a separate output that doesn't fall out of order-level scoring automatically.

What to do — this is a groupby, not new modeling:

```
vendor_risk_summary = orders.groupby('vendor_id').agg(
    avg_risk_score = mean(combined_risk_score),
    pct_orders_at_risk = mean(combined_risk_score > threshold),
    total_value_at_risk = sum(order_value where combined_risk_score > threshold),
    dominant_root_cause = mode(root_cause_category)
)
```

Repeat identically for `dc_id`, `lane_id`, `customer_id`, `sku_id`, `order_type`.

Why: gives category managers/ops leads a strategic view beyond one order at a time — e.g., "Vendor X: 34% of orders currently at-risk, \$180K value exposed, dominant cause: short-shipment" — the kind of insight that drives decisions like whether to keep or drop a vendor, not just whether to expedite one shipment.

Phase 8.7 — Service Impact Quantification

The gap this closes: translating risk scores into business-language dollar/service impact — the case brief's explicit "quantify service impact in terms of missed orders, lost revenue, customer penalties, service-level deterioration, or business priority impact" bullet, previously uncovered by the pipeline.

What to do:

missed_orders_value = SUM(order_value where otif_flag == 0 or combined_risk_score > threshold)
lost_revenue_estimate = at_risk_order_value × conversion_loss_rate # use a sourced/cited rate, never invented
penalty_exposure = order_value × penalty_rate # per customer tier if available
service_level_trend = current_period_OTIF% – prior_period_OTIF%
business_priority_impact = priority_score (risk_score × order_value × customer_tier_weight), surfaced explicitly

On sourcing the conversion_loss_rate rather than inventing it: external research indicates roughly 70% of customers who hit a stockout will buy from a competitor instead, and around 30% of those don't return — this can be cited directly rather than assumed, giving the impact calculation a defensible external basis instead of a made-up multiplier.

Note the important distinction from earlier in the build: customer penalty/order value data determines **severity/urgency** (how hard to push on a known risk), not **root cause** (what actually happened) — keep these two concerns structurally separate; conflating them makes both harder to explain cleanly.

12. Phase 9 — LLM Narrative Generation

What to do: A prompt template taking the fusion-layer record for one order (SHAP factors + BBN causal pathway + basic order context) and generating the plain-language explanation card.

Why: This is the layer that specifically demonstrates GenAI, distinct from the classical ML in Phases 4-7 — translating structured model output into something a non-technical planner reads and acts on in seconds.

Design guidance: template the tone/format tightly (one-line risk summary → top 2-3 causal factors in plain English → one recommended action) rather than letting the LLM free-write — this looks more production-ready and protects against odd generation appearing live in the demo video.

Honest scoping for the competition timeline: a well-structured Python template that *looks* LLM-generated is a legitimate, judge-respectable scope decision for a 2-week build, as long as you're upfront in Q&A that a production version would call a live API — claiming a live API is doing this work in the demo when it's actually templated would be an easy inconsistency for a judge to catch.

13. Phase 10 — Mitigation Mapping

What to do: A lookup table: `root_cause_category` → `recommended_action` (inventory shortage → reallocation/expedite; vendor delay → escalation/pre-drafted vendor message; DC delay → alternate carrier/DC rebalancing; transport issue → route/carrier change), gated through the Phase 8.3 conflict check before being finalized and attached to the order's explanation card.

Why: closes the loop from "here's a problem" to "here's the action" — explicitly required by the case brief's expected outcome list, and what turns this from a dashboard into a decision-support tool.

14. Phase 11 — Interactive Layer

Build three views on top of the fusion-layer output table:

- **Order lookup view:** text input for `order_id` → full card (risk score, SHAP factors, BBN causal pathway, plain-language narrative, conflict-checked recommended action)
- **Portfolio view:** sortable/filterable table of all open orders ranked by `combined_risk_score/priority_score`
- **Hotspot + Impact view** (*from Phases 8.5/8.7*): aggregate risk by vendor/DC/lane/customer/SKU/order-type, alongside the dollar/service-level impact summary

Why all three: the case brief asks for both "which future orders are at risk" (portfolio) and, per this build's own iteration, "give me the reason for one specific order" (lookup) and "highlight high-risk vendors/DCs/lanes/etc." (hotspot) — a real planner uses all three depending on whether they're doing daily triage, investigating a specific escalation, or making a strategic sourcing/vendor decision.

Tool: Streamlit — turns the pandas-based pipeline into a clickable web app with minimal extra code, fast enough to build in the available timeline, and what gets screen-recorded for the video.

15. Phase 12 — Validation and the Retraining/Feedback Loop

Validation: since the dataset is synthetic and self-authored, the "true" underlying risk factors are already known (e.g., which vendors were deliberately made unreliable). Check the model's outputs align with what was intentionally built — the deliberately declining vendors should show up as high-risk, derived root causes should make sense given the scenario constructed for that order. This is the only real internal-consistency check available before presenting, and it's a legitimate thing to state explicitly in the video ("we validated our model correctly identifies the specific risk patterns we engineered into the synthetic data") since it shows rigor without needing real client data.

Retraining/feedback loop — formalized as an explicit pipeline step, not just a narrative point: as open orders close, run them through Phase 1 to derive their actual root cause, append to the historical dataset, and periodically re-run Phases 3-6 (retrain the classifier, re-estimate BBN parameters). State a concrete cadence for the pitch (e.g., weekly retrain, daily scoring refresh) so it reads as a real operating rhythm rather than a one-time build.

16. Implementation Considerations (Slide 2 content — not a build phase)

This section answers the case brief's "highlight implementation considerations" bullet directly, using concrete references back to what's actually been built rather than generic statements:

- **Data quality** → the confidence flag from Phase 1, tied to which underlying system fields were actually complete for a given order
 - **Ownership of root-cause codes** → a named owner per category in a real deployment (procurement owns Vendor Failure, warehouse ops owns Warehouse Ops, etc.)
 - **Alert thresholds** → the recall-tuned decision cutoff from Phase 4, adjustable based on planner alert-fatigue tolerance
 - **Escalation governance** → the CONTESTED routing from Phase 8.3, sending resource-conflicted recommendations to a human decision-maker rather than auto-resolving
 - **Monitoring cadence** → the retraining schedule from Phase 12
-

17. Where to Actually Spend Limited Build Time

With four people and roughly two weeks, full working code for every phase is ambitious but divisible:

- **Highest priority, build fully working:** Phases 0-5 (data validation, root-cause derivation, feature engineering, XGBoost, SHAP) — standard, fastest to build, gives credible real output for the video.
 - **Second priority, can be partially simplified:** Phases 6-8 (BBN + fusion + the three 8.x additions) — even a simplified 5-7 node network with hand-set CPTs (skip full parameter learning if time is tight) still demonstrates the causal-pathway story; the conflict-check and rollup logic (8.3/8.5/8.7) are cheap groupby-style additions once order-level scores exist, so prioritize these over deeper BBN sophistication if time is short.
 - **Can be templated rather than live for the demo:** Phase 9 (LLM narrative) — legitimate scope management, disclosed honestly.
 - **Must be at least a working mockup:** Phase 11 (Streamlit interface) — this is what gets recorded, so it needs to look and feel real even if some backend logic is simplified.
-

18. Open Decisions Still Requiring a Team Conversation

1. **Open-order feature parity for secondary-cause-derived flags** (Phase 2) — the proxy-feature mapping between retrospective training flags and forward-looking scoring signals needs to be explicitly agreed before the scoring pipeline is built, not discovered after.
2. **Field list finalization with the teammates building the dummy dataset** — the full field list in Section 2 needs to be handed over before their generator is too far along, along with the request that failure patterns be built in deliberately (e.g., "~15% of orders have a genuine vendor short-shipment pattern") rather than fully random, since random noise gives the derivation rules and downstream model nothing real to learn from.
3. **Deliberately constructing at least one conflict-check demo scenario** (Phase 8.3) and one multi-cause order (Phase 1) — these need to be intentionally built into the synthetic data, not left to chance, since a valid pipeline can still fail to demonstrate these specific behaviors if the underlying data never actually produces the triggering conditions.